

Reducing SQIsign v2 Memory and Transferring Lifetime Scheduling to v3

An Order-Preserving Norm Sketch and RP2350 Evaluation

Hiro Nakanishi

Independent Researcher

quantumsity@protonmail.com

Preprint version: 5 September 2026

Abstract

Small SQIsign keys and signatures do not imply a small signing workspace. In the SQIsign v2.0.1 Level-I compact implementation studied here, five simultaneously live regions in `find_uv` total 6,339,868 bytes, exceeding RP2350 on-chip SRAM. We combine typed lifetime scheduling with an order-preserving norm sketch that stores a bit length and 64-bit prefix and regenerates full precision only when required.

For Level I/RADIX32, the operation arena falls from 353,008 to 172,080 bytes (51.2532%). The firmware uses no GMP, dynamic allocation, or external PSRAM; it reserves 321,408 of 532,480 SRAM bytes and leaves 211,072 unreserved. A 100-request differential- conformance test finds byte-identical output between the ordinary and low-memory APIs at the same commit; it is not a historical-response KAT. For the fixed image, linked call-graph analysis bounds the KeyGen, Sign, and Verify operation-PSP paths at 92,192, 120,664, and 20,980 bytes, including one 212-byte exception entry; handler frames, interrupt nesting, and live MSP use are excluded.

For v3.0, we overlay three lifetime-disjoint pairs in two functions. The official and adapted builds pass API, self-test, and official-response checks on 300 distinct parameter-set/request pairs each, giving 600 implementation-vector runs in total, and all six affected Arm frames shrink. One commit-pinned `p324_3` RP2350 path reduces the Sign PSP watermark from 101,060 to 96,396 bytes. These target data cover one parameter and one deterministic path, not a timing distribution or whole-program stack bound. Software timing and trace tests find input-dependent behavior; we make no claim of constant-time or power/EM resistance.

Keywords: SQIsign, embedded cryptography, memory optimization, lifetime scheduling, microcontroller, side-channel analysis

1 Introduction

SQIsign is a Round-3 candidate in NIST’s additional post-quantum signature process[1, 2]. It obtains unusually small public keys and signatures from supersingular-isogeny and quaternion computations [3, 4, 5]. This communication compactness is attractive for constrained systems, but the signing path still combines quaternion arithmetic, four-dimensional lattice routines, norm equations, candidate enumeration, and large-integer operations.

Fixed-precision arithmetic removes unbounded integer growth, but replacing a heap object with a maximum-sized stack array does not by itself reduce total SRAM. In our v2 starting point, five live allocations in one `find_uv` invocation total 6,339,868 bytes, about 11.9 times the RP2350’s 532,480-byte SRAM. The central systems question is therefore which values must coexist, not merely which allocator stores them.

This paper studies four questions:

- Q1. Can a complete deterministic SQIsign v2 Level-I KeyGen/Sign/Verify path run in RP2350 on-chip SRAM without GMP, a heap, or external PSRAM?
- Q2. Can the resident table of full-precision candidate norms be removed while preserving the reference comparison and search order?
- Q3. What are the accompanying SRAM, stack, flash, time, and side-channel observations, and which conclusions do they support?
- Q4. Does lifetime scheduling remain useful after the substantial v3 redesign, where the v2 candidate table no longer exists?

Our contributions are as follows.

1. We formulate protocol workspace placement as an aligned storage allocation problem over typed objects and path-sensitive lifetimes. Reviewed lifetime annotations are translated into C unions, offsets, and static assertions.
2. We introduce an order-preserving norm sketch for the v2 candidate table. It reduces resident state while retaining a full-precision fallback whenever the sketch alone cannot decide an ordering.
3. We validate the v2 construction with source-correspondence and lifetime audit records, sanitizer and guard tests, ELF audits, two fresh-process repetitions of a 100-request differential experiment, fixed-image operation-PSP analysis, and RP2350 measurements.
4. We transfer lifetime scheduling, but not the v2 norm sketch, to two v3 functions. We test all three official parameter sets on the host and in Arm object builds, and report one commit-pinned p324_3 target path.
5. We evaluate implementation security separately from functionality and memory. The measured implementations remain variable-time, and no analog trace experiment has been performed.

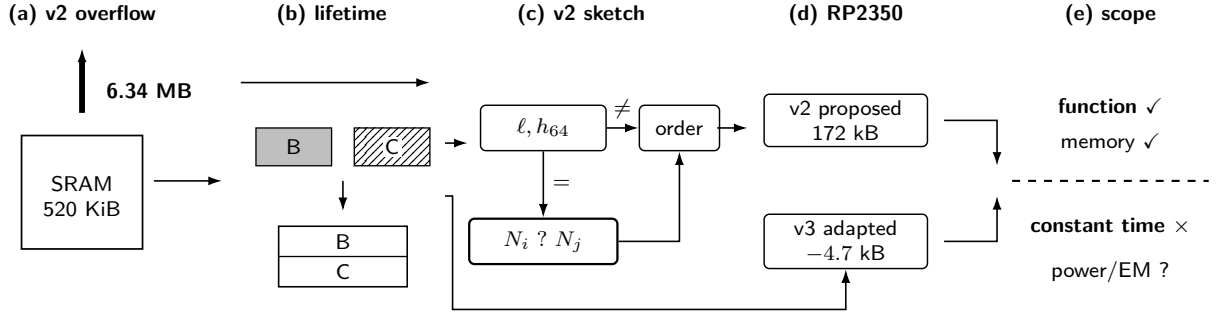


Figure 1: Study overview. The v2 path combines lifetime scheduling with an order-preserving norm sketch. The v3 two-function variant transfers only the scheduling principle. Functionality and memory evidence are evaluated separately from side-channel behavior.

2 Related Work

Reusing storage for noninterfering lifetimes is an established compiler and memory-allocation principle. Graph-coloring register allocation represents simultaneously live values by interference edges[6], and compile-time memory allocation has considered differently sized objects [7]. We do not claim to invent lifetime-based reuse. Our focus is its application to complete SQIsign operation closures, including typed layout, failure paths, clearing, linked-image accounting, and a reconstructible evidence chain.

Fixed-precision SQIsign first supplied uniform bounds of 7026/10713/14150 bits for Levels I/III/V[8]. Compact Quaternion Algorithms later reduced these values to 1665/2521/3319 bits and improved host performance [9]. Qlapoti and follow-up work improve the ideal-to-isogeny translation and its analysis [10, 11, 12]. These advances bound or accelerate arithmetic objects; our v2 contribution instead targets their simultaneous residency across the protocol.

For embedded SQIsign, pqm4 documented the difficulty of integrating the v2 signing side because of GMP and dynamic allocation[13]. Cortex-M work has optimized finite-field arithmetic or one-dimensional verification[14, 15], while a complete KeyGen/Sign/Verify implementation has been reported for larger Arm systems [16]. Other nearby implementations include a 32-bit deployment study[17], an AVX-512 implementation [18], and a pure-Rust v2 implementation that still uses heap-backed integers[19]. The v3 submission itself now includes complete Cortex-M4 paths without external GMP [5, 20]. Accordingly, the v2 on-chip result and the v3 transfer experiment are reported as distinct contributions, not as a performance comparison between generations.

Constant-time integer, lattice, and quaternion components for SQIsign are active research topics[21, 22, 23]. Mukherjee et al. demonstrated a simple-power-analysis key-recovery route via a secret-derived exponent in Cornacchia processing[24]. These works show why a static address layout is not a side-channel countermeasure. Our timing and software-trace tests identify residual variable behavior; they do not replace power/EM acquisition or attack reproduction.

A documented search through 2026-09-04, after the v3 release, found no prior report combining the following v2 properties: complete KeyGen/Sign/Verify, no GMP or heap, on-chip-only RP2350 execution, and linked-image accounting. The public search log records the queries, databases, inclusion criteria, and nearby counterexamples. This observation is limited to the documented search and does not cover unpublished or later work.

3 Background and Evaluation Scope

3.1 SQIsign v2 and v3

The v2.0.1 implementation studied here uses a two-dimensional response path, including a chain of (2,2)-isogenies, and is not the earlier one-dimensional verification setting[4]. We fix Level I and the RADIX32 finite-field backend. Quaternion integers use 27 64-bit limbs: a 1665-bit magnitude bound plus sign storage rounded to a 1728-bit array. Multiplication, division, GCD, square root, and modular exponentiation operate on these fixed-capacity arrays and caller-owned workspaces.

SQIsign v3.0 changes parameters, integer representation, lattice reduction, ideal representation, ideal-to-isogeny conversion, and response sampling [5]. Its official source provides complete Cortex-M4 KeyGen, Sign, and Verify paths[20]. The v2 `find_uv` candidate table is absent from the v3 path, so the norm sketch has no direct v3 target. Large, sequentially used local states remain, which motivates the separate lifetime experiment in section 6.

3.2 Why fixed-capacity integers

GMP and its smaller mini-GMP implementation represent integers with separately allocated limb arrays and grow those arrays through an allocator [25]. Mini-GMP passed the functional response checks in a separate host experiment, but a simple 16-byte-aligned static-pool allocator needed 317,696 bytes for the measured 100-vector signing test set and saw up to 4,830 live blocks. This host threshold is neither a mini-GMP lower bound nor an RP2350 total. We choose fixed-capacity arrays because their ownership, alignment, and reservation can be audited before linking, not because fixed capacity implies constant-time arithmetic. Used-limb scans, Euclid loops, comparisons, and exponentiation remain variable-time.

3.3 Target and memory quantities

The target is a Raspberry Pi Pico 2 with one Arm Cortex-M33 core of its RP2350, clocked at 150 MHz[26]. Code and read-only data execute from internal flash; only the chip’s 532,480 bytes of SRAM are counted as available data memory. The builds use Pico SDK 2.3.0 and Arm GNU Toolchain 15.2.1 in Release mode.

We distinguish five quantities: operation-arena payload, guarded operation region, linked mutable regions, reserved Process/Main Stack Pointer regions, and an observed stack watermark. A watermark is the overwritten extent for one execution, not a bound. “Unreserved SRAM” is computed after subtracting the full PSP and MSP reservations, never only the observed watermarks.

3.4 Security evaluation scope

Functional failures include output disagreement, boundary-canary corruption, uncleared workspace state, or an unintended allocator/GMP dependency. For side channels, we consider an observer of time, control flow, addresses, power, or electromagnetic emanations. This study records host timing and software traces and selected RP2350 timing diagnostics. It records no full-Sign power or EM trace. Constant-time execution, masking, fault resistance, production randomness, and resistance to key recovery are not claimed.

4 Lifetime Scheduling Model

Let $O = \{o_1, \dots, o_n\}$ be the operation objects and $\mathcal{T}$ the set of allowed traces, including retry and failure returns. Object o_i has size s_i , alignment a_i , type τ_i , and a secrecy attribute. Its live points on trace t are $L_i(t)$. We define the interference graph $\mathcal{G} = (O, E)$ by

$$(o_i, o_j) \in E \iff \exists t \in \mathcal{T} : L_i(t) \cap L_j(t) \neq \emptyset. \quad (1)$$

Thus, disjointness on one successful execution is insufficient.

For offsets $x_i \geq 0$, a valid typed placement satisfies

$$x_i \equiv 0 \pmod{a_i}, \quad (2)$$

$$(o_i, o_j) \in E \implies (x_i + s_i \leq x_j) \vee (x_j + s_j \leq x_i). \quad (3)$$

The arena height is $M_{\text{arena}} = \max_i(x_i + s_i)$. We do not solve the general optimization problem. Reviewed phase annotations yield a feasible first-fit placement, emitted as typed C unions/structures with `sizeof`, `offsetof`, alignment, and static-assert checks.

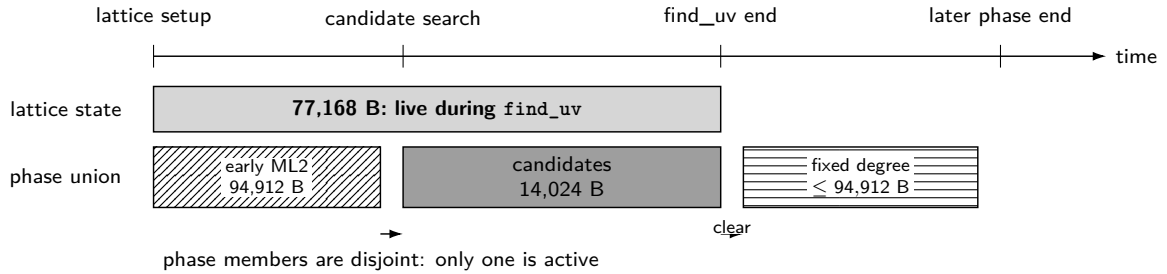
For pre/post guards $g_{\text{pre}}, g_{\text{post}}$, PSP and MSP reservations $R_{\text{PSP}}, R_{\text{MSP}}$, and other exclusive mutable state R_{other} , the firmware quantity is

$$R_{\text{excl}} = (M_{\text{arena}} + g_{\text{pre}} + g_{\text{post}}) + R_{\text{PSP}} + R_{\text{MSP}} + R_{\text{other}}. \quad (4)$$

For v2 these terms are 172,080, 128, 131,072, 8,192, and 9,936 bytes, giving 321,408 bytes.

Two objects may share an offset only when their reviewed contracts establish: (i) disjoint lifetimes on success, retry, and failure paths; (ii) no pointer escape or returned reference; (iii) required clearing before reuse; (iv) typed aliasing and alignment; and (v) cleanup on every return edge. ABI probes, `-Wvla -Walloca -Werror`, ASan/UBSan, canaries, zeroization tests, and ELF audits provide complementary checks. They do not constitute whole-program alias verification.

(a) Logical lifetimes



(b) Physical layout



Figure 2: v2 logical lifetimes and their typed physical layout. The 77,168-byte lattice state overlaps none of the `find_uv` phases. Three mutually exclusive phase members share the 94,912-byte phase union.

KeyGen, Sign, and Verify also share one operation arena because the embedded API serializes them. This makes the reservation their maximum rather than their sum. Reentrancy, concurrent signing, and cross-core use are outside the API contract.

5 The v2 Method

5.1 Reducing resident state

The starting `find_uv` stores candidate vectors, full-precision norms, quotients, row counts, and sorting records simultaneously. We apply the following transformations in sequence, checking differential transcripts after each transformation:

1. pack each four-coordinate candidate in `int8_t[4]` after verifying that each coordinate lies in $[-2, 2]$;
2. sort a `uint16_t` index permutation rather than candidate copies;
3. reconstruct per-candidate quotients at their use sites;
4. stream two rows instead of retaining all seven;
5. move large variable-length locals into caller-owned typed workspaces;
6. overlay ML2, candidate, fixed-degree, theta, discrete-log, inversion, and integer-temporary phases where the lifetime contracts permit it;

7. share the serialized KeyGen/Sign/Verify owner; and
8. replace the two full-precision norm rows with compact sketches and a full-precision fallback.

Table 1: v2 ablation. Rows before explicit lattice ownership describe the candidate region; later rows describe caller-owned workspace or the operation arena, and are not maximum memory values for the complete firmware.

Configuration	Region [B]	Change introduced
Compact starting point	6,339,868	Five simultaneous heap payloads
Packed coordinates	2,044,252	Four-byte candidates and packed records
Index sorting	1,905,728	Reusable <code>uint16_t</code> permutation
Deferred quotient	962,240	Reconstruct at use sites
Two-row streaming	275,840	Retain two of seven rows
Explicit lattice owner	353,008	Move a 77,168-byte VLA into the owner
Full-norm-table baseline	353,008	Overlay other phases; no dynamic Arm frame
v2 proposed	172,080	Sketch rows plus full-precision fallback

Moving the lattice VLA to an owner intentionally increases the displayed region from 275,840 to 353,008 bytes. This accounting treatment prevents a change of storage class from being misreported as reduced total memory.

5.2 Order-preserving norm sketch

For a candidate v , the positive integer norm $N(v)$ can be reconstructed from its retained packed coordinates, the row Gram matrix G_j , and adjusted norm d_j . Let

$$\ell(n) = \begin{cases} 0, & n = 0, \\ \lfloor \log_2 n \rfloor + 1, & n > 0, \end{cases} \quad (5)$$

and, for $w = 64$,

$$h_w(n) = \begin{cases} n, & \ell(n) \leq w, \\ \lfloor n/2^{\ell(n)-w} \rfloor, & \ell(n) > w. \end{cases} \quad (6)$$

The resident sketch is $S_w(n) = (\ell(n), h_w(n))$. Candidate comparison uses bit length, the retained 64-bit prefix, reconstructed full-precision norms on a sketch tie, and the original enumeration index, in that order.

Proposition 1 (Order preservation). *Suppose $N(v_i)$ can be reconstructed with full precision for every retained candidate. The sketch comparator orders candidates as the lexicographic key $(N(v_i), i)$ does.*

Proof. Integers with different bit lengths have the same order as their lengths. For equal lengths and unequal 64-bit prefixes, the most significant differing bit lies in a retained position, so the prefix decides the integer order. If both sketch fields agree, the comparator reconstructs and compares the two full-precision integers. It uses the enumeration index only when those integers are equal. These cases cover every pair. $\square$

For example, 2^{100} and $2^{100} + 1$ have equal sketches but are sent to the reconstruction path. Correctness therefore does not depend on assuming that sketch ties are rare. In the attribution test set, reconstruction was used for 5,041 of 2,062,361 sorting comparisons (0.2444%); this frequency is a performance observation only.

Algorithm 1 gives a compact, self-contained view of the two central changes. `PREPAREROW` generates candidates in the reference order and sorts their index permutation with `COMPARE`. `SEARCHPAIRS` retains the reference row-pair, in-row, and congruence orders. The detailed source obligations and failure cases appear in table 2.

Algorithm 1 Order-preserving comparison and two-row `find_uv` schedule

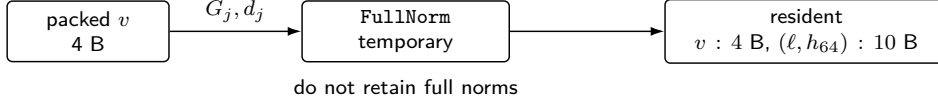
Require: packed candidates, row descriptors $R_j = (G_j, d_j)$, target T , and caller-owned workspace W

Ensure: reference-key comparison; FINDUV-LM clears W on every exit

```
1: function COMPARE( $i, j, V, S, G, d$ )
2:   if  $i = j$  then
3:     return 0
4:   else if  $S[i].\ell \neq S[j].\ell$  then
5:     return CMP( $S[i].\ell, S[j].\ell$ )
6:   else if  $S[i].h \neq S[j].h$  then
7:     return CMP( $S[i].h, S[j].h$ )
8:   end if
9:    $n_i \leftarrow \text{FULLNORM}(V[i], G, d)$ ;  $n_j \leftarrow \text{FULLNORM}(V[j], G, d)$ 
10:  if  $n_i = \text{FATAL}$  or  $n_j = \text{FATAL}$  then
11:    return FATAL
12:  else if  $n_i \neq n_j$  then
13:    return CMP( $n_i, n_j$ )
14:  else
15:    return CMP( $i, j$ )
16:  end if
17: end function
18: function FINDUV-LM( $T, \mathcal{I}, W$ )
19:   $(s, L, R_0, \dots, R_{r-1}) \leftarrow \text{PREPARELATTICE}(T, \mathcal{I}, W)$ 
20:  if  $s = \text{FATAL}$  then
21:    return FINISH(FATAL,  $W$ )
22:  end if
23:  CLEARPHASESTORAGE( $W$ )
24:  for  $j \leftarrow 0$  to  $r - 1$  do
25:     $c[j] \leftarrow \text{PREPAREROW}(j, 0, W)$  ▷ validation pass
26:    if  $c[j] = \text{FATAL}$  then
27:      return FINISH(FATAL,  $W$ )
28:    end if
29:  end for
30:  for  $j_1 \leftarrow 0$  to  $r - 1$  do
31:     $m_1 \leftarrow \text{PREPAREROW}(j_1, 0, W)$ 
32:    if  $m_1 \neq c[j_1]$  then
33:      return FINISH(FATAL,  $W$ )
34:    end if
35:    for  $j_2 \leftarrow j_1$  to  $r - 1$  do
36:      if  $j_2 = j_1$  then
37:         $(V_2, m_2) \leftarrow (W.V[0], m_1)$  ▷ diagonal alias
38:      else
39:         $m_2 \leftarrow \text{PREPAREROW}(j_2, 1, W)$ ;  $V_2 \leftarrow W.V[1]$ 
40:        if  $m_2 \neq c[j_2]$  then
41:          return FINISH(FATAL,  $W$ )
42:        end if
43:      end if
44:       $z \leftarrow \text{SEARCHPAIRS}(T, W.V[0], V_2, R_{j_1}, R_{j_2})$ 
45:      if  $z = \text{FATAL}$  then
46:        return FINISH(FATAL,  $W$ )
47:      end if
48:      if  $z$  is a solution then
49:        if not VALIDATEANDPUBLISH( $z$ ) then
50:          return FINISH(FATAL,  $W$ )
51:        end if
52:        return FINISH(SUCCESS,  $W$ )
53:      end if
54:    end for
55:  end for
56:  return FINISH(NOTFOUND,  $W$ )
57: end function
```

Here FULLNORM evaluates the full-precision quadratic form, requires a positive quotient with zero remainder on division by d , and otherwise returns FATAL. FINISH clears the complete caller-owned workspace before returning.

(a) Construction



(b) Comparison

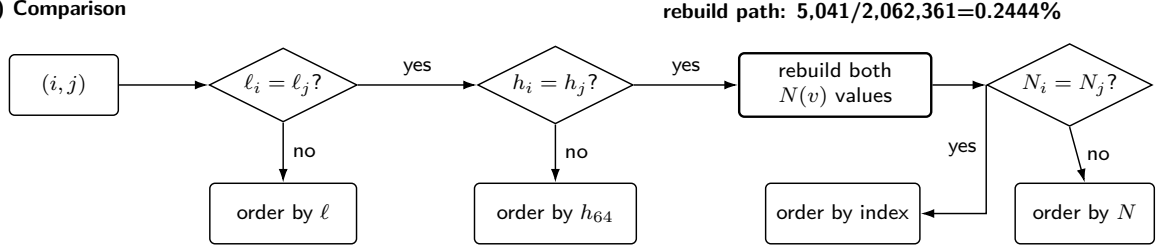


Figure 3: Resident sketch and comparison path. A tie causes reconstruction of both full-precision norms before the comparator returns an order.

Rows contain at most 624 candidates and are sorted by an in-place heapsort of their index permutation. If norm reconstruction fails, the comparison callback latches a fatal status; an index order is returned only to finish the sort, after which the entire row is rejected. The temporary order is never published as a successful result.

5.3 From the proposition to the C implementation

The full-norm-table reference is fixed at commit 6b79cfb5.... The proposed commit 71099e08... is its direct child. A source audit binds 22 predicates and anchors to their digests. Table 2 summarizes the obligations connecting proposition 1 to the C search.

Table 2: Correspondence between the v2 full-norm-table reference and the proposed `find_uv`.

Obligation	Reference	Proposed	Failure handling
Row generation	Same coordinate order, sign representative, 2/3 filters, order-4 representative, odd norm	Same loops and predicates, followed by <code>finduv_eval_norm</code>	Nonintegral division, nonpositive value, or capacity overflow is fatal
Sort key	Full-precision $(N(v), i)$	Length, top 64 bits, reconstructed norm, then i	Reconstruction error is latched and rejects the row
Row pairs	Increasing j_1 , then $j_2 \geq j_1$	Same triangular order; diagonal aliases one row	A regenerated count mismatch is fatal
In-row order	Increasing i_1 ; $i_2 = i_1$ on diagonal and 0 otherwise	Same starts and increments	Norm reconstruction error is fatal
Congruence sequence	Same initial residue, stride n_1 , and $v < \lfloor T/n_2 \rfloor$	Same inverse, bound, and loop order	Noninvertible continues; damaged arithmetic is fatal
Publication	Validate and publish only after success	Same publication structure; Success/NotFound/Fatal remain distinct	Common terminal cleanup clears the workspace

Proposition 2 (First accepted result). *For a fixed input, assume arithmetic succeeds in both implementations and the correspondences in table 2 hold. The proposed search returns the same first accepted tuple as the reference, or both return NOTFOUND within the reference search domain.*

Proof. By proposition 1, each reconstructed row has the reference candidate order. A validation pass retains only row counts, and the search pass reconstructs rows deterministically. The row-pair, in-row, and congruence orders are the same by table 2; hence both searches first accept the same tuple or exhaust the same sequence. $\square$

This result is conditional on the source correspondence at those commits. It is not a formal proof of all C behavior. The source audit is therefore combined with 12 reference/proposed transcripts, the two 100-request differential runs, an intentional sketch-tie test, and NotFound, retry, fatal, guard, and clearing tests.

5.4 Lifetime audit records and linked-code scope

The evaluated v2 firmware fixes 52 project translation units reachable from the Level-I/RADIX32 operations. ELF checks reject allocator, GMP, system-RNG, legacy large-stack, unresolved-symbol, or out-of-region sections; the linked heap is zero bytes. Eleven principal regions have a machine-readable lifetime audit

record describing type, Arm size/alignment, offset, construction and last-use anchors, prohibited co-residents, pointer passing and escape review, cleanup on success/failure/retry, and associated static/dynamic tests.

Table 3: Summary of the principal v2 lifetime audit records.

Object	Size [B]	Align [B]	Exclusion contract	Exit checks
Operation arena	172,080	8	Serialized KeyGen/Sign/Verify	Full zero scan and guards
KeyGen union	172,080	8	<code>find_uv</code> versus discrete log	Terminal clear, retry contract, assertions
Sign union	172,080	8	Seven principal phase members	Terminal clear, guards, 100-request tests
<code>find_uv</code> lattice	77,168	8	Coexists with candidates; no overlay	Offset assertions and ASan/UBSan
<code>find_uv</code> phase union	94,912	8	Lattice multiply, candidate, fixed degree	One-way transitions and all terminal exits
Verify union	15,428	4	Even-isogeny versus theta	Wrapper cleanup and modified-signature test

The pointer-escape scan rules out direct global/static stores in the audited source, but is not a sound whole-program alias analysis. In particular, an outer Sign retry clears/finalizes the active callee and relies on subsequent initialization; a full-owner zero scan is performed at the public operation’s terminal exit. The audit record preserves this distinction rather than silently treating every retry as a whole-owner clear.

6 Transfer to SQIsign v3

The official v3 `p324_3/m4f` object reports a 34,816-byte frame for `quat_l1l_dual_reduce_ideal`. We call the implementation with changes only to this function the *one-function variant*. It introduces two typed overlays:

1. lattice-reduction state `fp_ldl_t` shares storage with the integer matrix `Aout`, which is first constructed after reduction; and
2. `Ainv_total`, after deriving `Aout`, shares its equal sized region with the later output basis `Bout`. The still-live Gram matrix and transform do not overlap it.

The *two-function variant* adds a third overlay in `protocols_sign`: the commitment ideal’s last reference precedes the first write of the challenge-isogeny output product. Source-anchored audit records check those boundaries and the equal type sizes. The two-function variant changes ownership and placement in two functions; it does not change the API, algorithms, branch conditions, or randomness consumption. Its source commit is `874658c6...`, based on official v3 commit `6d017708...`. The RP2350 firmware is tied to harness commit `5df7acfb...`; the reported build contains no uncommitted source or harness changes. Reconstruction patches and SHA-256 bindings for the generated source, binary, and target record are public.

This transfer remains local. It does not establish an automatic whole-v3 placement, and the two-function image retains 19 compiler-reported dynamic frames. A separate *fixed-frame analysis build* is used to study stack bounds; its results do not transfer to the two-function binary.

7 Evaluation Method

7.1 Version isolation

Each executable is built independently from a pinned source commit without sharing object files across variants. Within-v3 comparisons use the same official base, parameter set, toolchain, and options. Cross-generation timing or PSP differences are descriptive only because parameters, algorithms, integer backends, and memory ownership change together. In particular, the explicit v2 arena and the v3 PSP watermark are not subtracted as though they were the same resource.

7.2 Functional validation

For v2, 12 fixed transcripts compare the full-norm-table and proposed implementations. One ordinary-API process and two separately started low-memory processes each consume all 100 official v2 Level-I requests. Each vector checks public/secret keys, signed message, post-RNG state, successful Open, one-bit modified-signature rejection, guards, and terminal clearing. All three generated response streams are byte-identical. The expected response is produced by the ordinary path at the same compact-source commit. The two

low-memory runs therefore test fresh-process repeatability, but do not provide independent oracles. This is a differential-conformance test using official requests, not a historical-response KAT.

For v3, official and two-function builds each pass the NIST API test, scheme self-test, and 100 official-response vectors for p324_3, p500_27, and p664_17. This is 300 distinct parameter-set/request pairs run once on each implementation, or 600 implementation-vector runs in total. The one-function variant additionally has five interleaved official/adapted target pairs and a 10-vector experiment in two linked-code placements. The two-function variant’s target evidence is one commit-pinned p324_3 vector-0 KeyGen/Sign/Verify run. These test sets are finite regression tests, not all-input equivalence proofs.

7.3 Stack method

The v2 stack analysis uses 126 GCC stack-usage files with 1154 records and no dynamic record. It resolves every linked, input-dependent Thread-mode path from the three operation roots, including indirect calls and manually bound assembly/runtime frames. Two recursive components halve their length at each child; with maximum length 248 they have at most nine active frames. A source, constant, object, or function-body digest change stops the audit.

The longest condensed-call-graph path receives a conservative 212-byte allowance for one maximum Secure Armv8-M exception entry [27]. The result concerns the operation PSP. Handler software uses the MSP and is not included; neither are the enabled-interrupt set, nesting policy, or live MSP at entry.

The separate v3 fixed-frame analysis build follows the same pattern after replacing 19 dynamic frames with fixed-capacity frames. It closes the linked operation roots, but retains 18 indirect callback sites in candidate interrupt-handler paths. Hence neither the v2 nor fixed-frame result is a whole-program, interrupt-inclusive stack bound.

8 Results

8.1 v2 SRAM and flash

The two full-precision norm rows occupy 269,568 bytes. Removing them reduces the candidate workspace from 275,840 to 14,024 bytes. The phase union stops at 94,912 bytes because an ML2 retry workspace becomes the largest member. Together with the 77,168-byte long-lived lattice state, the complete operation arena is therefore

$$77,168 + \max(94,912, 14,024) = 172,080 \text{ bytes.} \quad (7)$$

The full-norm-table baseline is 353,008 bytes, so the reduction is 180,928 bytes, or 51.2532%.

Table 4: Linked v2 proposed firmware memory accounting.

Quantity	Bytes	Scope
On-chip SRAM total	532,480	main + scratch
Operation-arena payload	172,080	KeyGen/Sign/Verify union
Guarded operation region	172,208	main bank
PSP reservation	131,072	main bank
Other main-bank state	9,936	data and runtime
Main-bank linked use	313,216	preceding rows
MSP reservation	8,192	scratch bank
Exclusive SRAM reservation	321,408	−180,928 B
Unreserved SRAM	211,072	+180,928 B
Heap	0	no allocator symbol
BIN image	288,384	+880 B

The arena reduction propagates one-for-one to exclusive SRAM and unreserved SRAM; it is not a move from heap to stack. The proposed BIN image is 288,384 bytes, 880 bytes larger than the baseline, reflecting the reconstruction and comparison code.

8.2 v2 operation PSP and time

Two boots of the same stored UF2 and deterministic input observe PSP extents of 91,980, 120,452, and 20,768 bytes for KeyGen, Sign, and Verify. They also reproduce the transcripts and stack extents. This is a cross-check, not input-population sampling.

Table 5: v2 operation-PSP analysis for the fixed image. The rightmost column is relative to the 131,072-byte reservation.

Operation	Observed [B]	Software path [B]	PSP result [B]	Margin [B]
KeyGen	91,980	91,980	92,192	38,880
Sign	120,452	120,452	120,664	10,408
Verify	20,768	20,768	20,980	110,092

The smallest PSP margin is 10,408 bytes for Sign. The analysis covers all linked input-dependent Thread-mode paths in this image plus one maximum exception entry, but excludes handler frames, nesting, and MSP usage.

Table 6: First RP2350 boot for the deterministic v2 operations. Cycles are elapsed microseconds multiplied by 150, not hardware-counter readings.

Operation	Time [s]	Cycles at 150 MHz	PSP extent [B]
KeyGen	2,698.150029	404,722,504,350	91,980
Sign	7,611.258527	1,141,688,779,050	120,452
Verify	0.814341	122,151,150	20,768

KeyGen takes about 45 minutes and Sign about 127 minutes, so the v2 result is a memory-feasibility result rather than a real-time signer. On the host, two reversed-order 30-row experiments give proposed/baseline median ratios of 1.003695 and 1.006464 for KeyGen, and 0.999481 and 0.998980 for Sign. A small KeyGen cost is visible; a Sign slowdown direction is not reproduced. Single target observations have ratios 1.000704, 1.037312, and 1.000593, but cannot estimate a target timing distribution. The design therefore trades an 880-byte flash increase and small local-frame increases for a large RAM reduction; it is not a claim of improvement on every metric.

8.3 v3 local transfer

In the one-function variant, the two overlays reduce the `quat_111_dual_reduce_ideal` frame by 3,928 bytes; five paired target runs show the same reduction in Sign PSP. The two-function variant adds the second function and produces the results in table 7.

Table 7: Official v3 versus the two-function variant on `p324_3`. Frame sizes are Arm object values; official PSP is the earlier five-run median and the adapted PSP is one observation from the fixed build.

Quantity	Official v3	Two-function	Difference
<code>quat_111_dual_reduce_ideal</code> frame [B]	34,816	30,888	-3,928
<code>protocols_sign</code> frame [B]	20,008	19,272	-736
KeyGen PSP extent [B]	56,972	56,972	0
Sign PSP extent [B]	101,060	96,396	-4,664 (-4.6151%)
Verify PSP extent [B]	37,444	37,444	0

The two-function Sign PSP watermark is 96,396 bytes, a 4,664-byte (4.6151%) reduction from official v3. The incremental 736-byte reduction from the one-function to the two-function variant equals the `protocols_sign` frame reduction. The two-function variant has only one target run, so its elapsed time is reported descriptively and not used for a slowdown estimate.

All six function-frame comparisons decrease, while all six host implementation/parameter combinations pass their 100 official-response vectors. This supports applicability of the three local overlays across the official parameter sets. It does not supply whole-image SRAM or target watermarks for `p500_27` or `p664_17`.

The one-function 10-vector/two-placement experiment passes 40 valid KeyGen/Sign/Verify trials and 40 modified-signature rejections. PSP extents agree across both code placements, and the 3,928-byte Sign reduction appears in all 20 observations. It addresses a one-vector/one-address alternative explanation for the one-function result, but it is not a multi-input experiment on the two-function variant.

8.4 Fixed-frame stack-bound experiment

The fixed-frame analysis build converts the 19 dynamic-frame reports to fixed-capacity frames, rejects capacity violations with fail-stop traps, and builds with VLA/alloca errors. The linked KeyGen/Sign/Verify

Table 8: Two-function-variant Arm frame changes for all official v3 parameter sets.

Parameter	Function	Official [B]	Adapted [B]	Reduction
p324_3	quat_111_dual_reduce_ideal	34,816	30,888	-3,928 (11.2822%)
p324_3	protocols_sign	20,008	19,272	-736 (3.6785%)
p500_27	quat_111_dual_reduce_ideal	46,088	40,904	-5,184 (11.2480%)
p500_27	protocols_sign	27,304	26,328	-976 (3.5746%)
p664_17	quat_111_dual_reduce_ideal	63,016	55,904	-7,112 (11.2860%)
p664_17	protocols_sign	36,728	35,392	-1,336 (3.6376%)

operation-PSP results, after adding the 212-byte exception entry, are 108,300, 127,932, and 40,468 bytes. A commit-pinned vector-0 target run observes 62,096, 101,060, and 40,252 bytes, respectively.

This is a negative result for the simple hypothesis that replacing VLAs with fixed arrays automatically lowers measured stack: the analysis build leaves the Sign watermark unchanged and increases KeyGen and Verify. More importantly, the analysis build is a different binary from the two-function variant. Its operation-PSP analysis says nothing about that variant’s 19 dynamic frames, handler callbacks, IRQ nesting, or MSP use.

Table 9: Evidence included in each stack result and the scope excluded from it. The fixed-frame analysis build and two-function variant are different binaries.

Layer	Available evidence	Excluded scope
v2 KeyGen/Sign/Verify operation PSP	Linked input-dependent Thread-mode paths plus one maximum PSP exception entry	Handler software, nesting, and entry-time MSP
v3 fixed-frame operation PSP	Linked software paths plus the same exception allowance	Transfer to the two-function binary and handler/MSP behavior
v3 two-function operation PSP	Two local frame changes and one p324_3 watermark	An operation bound for the 19 dynamic frames
Interrupt-side stack	MSP reservation/watermark and direct-callee metadata checks	Enabled IRQs, indirect callbacks, nesting, and entry-time MSP

9 Side-Channel Findings

A fixed arena removes allocator-address variation but does not make loop counts, branches, or memory accesses independent of secrets. The v2 path still contains sketch-tie reconstruction, variable candidate search, rejection, used-limb scans, early-return comparison, Euclid loops, and secret-derived modular exponents.

For v2, two reversed-order fixed-versus-random host timing tests, 64 samples per class each, give first-order statistics 3.570 and 3.576 and centered second-order statistics -5.410 and -5.374. Both second-order values cross the commonly used $|t| > 4.5$ exploratory threshold[28]. This small test detects a repeatable distributional difference; a value below the threshold would not certify nonleakage. Full-Sign software traces also show reproducible edge and effective-address differences when signing randomness changes.

The known SPA route targets secret-derived exponents during Cornacchia processing[24]. In our 12-seed v2 test set, Sign reaches the Cornacchia modular-square-root path 361 times, 190 in the modulus class that directly matches the published exponent relation. An isolated RP2350 powmod diagnostic has correlation 0.999860 between exponent Hamming weight and time, with an approximate slope of 4.004 ms per set bit. This is a timing diagnostic, not an analog trace or a full-Sign key-recovery experiment.

For v3, a fixed-message, fixed-signing-randomness, fixed-key-address test uses ten official keys in two key orders. Across official and one-function host builds, 1,200 verified signatures reproduce key-associated median rankings; the minimum between-order Spearman coefficient is 0.987879. The corresponding RP2350 experiment collects 200 verified signatures with cache normalization; both implementations have rank coefficient 1.000000 and key-median spans of 50.8419–51.3752%. Public and secret key components change together, so the result is key-associated timing, not secret-only attribution.

Fixed-schedule exponentiation and sampling prototypes remove selected local variations but leave value-dependent or upper-level work. No prototype meets a deployment criterion. These measurements therefore show residual variability and do not support a side-channel-resistance claim; memory reduction and response-vector agreement do not alter that conclusion.

Available evidence

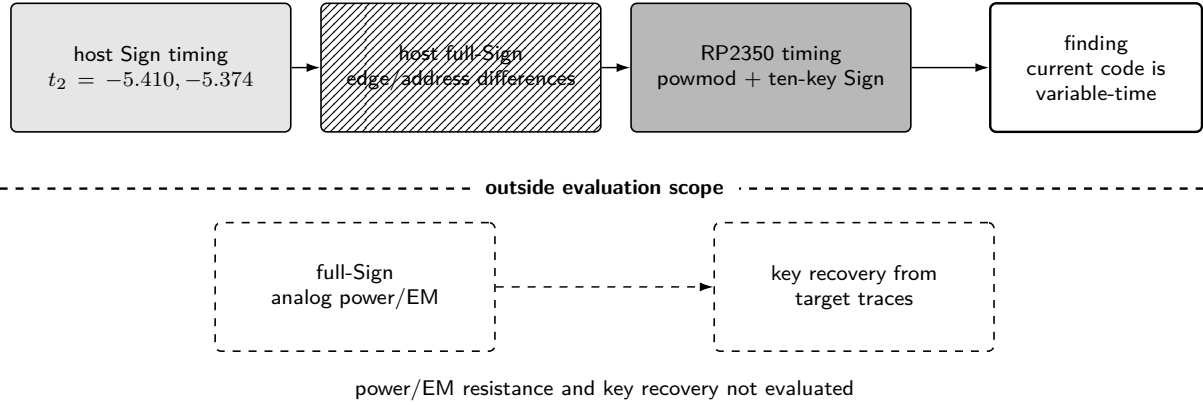


Figure 4: Scope of the side-channel evidence. The available experiments detect variable behavior. Full-Sign analog acquisition and target-trace key recovery remain unperformed.

10 Discussion

10.1 Transferability and RAM–stack–flash–time trade-offs

The common element across v2 and v3 is the procedure for identifying typed objects, establishing noninterference from their lifetimes, and making the resulting storage reuse explicit in C. The v2 norm sketch, 1665-bit integer bound, and 172,080-byte arena are version-specific. The v3 results show that the placement procedure applies to different local states, but do not establish generality beyond the two SQIsign functions and three parameter sets tested here.

The v2 firmware leaves 211,072 bytes unreserved, compared with 30,144 bytes for the full-norm-table baseline. This is integration capacity rather than demonstrated capacity for a particular combination of randomness, communications, interrupts, and countermeasures. Constant-time or masked implementations may require additional state and fresh randomness, so their memory and security costs must be evaluated together. For v3, only local stack changes were measured; an exclusive-SRAM total comparable to the v2 accounting is not available.

10.2 Annotation-driven layout generation

Each layout annotation specifies phases, size, alignment, secrecy, pointer-escape status, and clearing events. The generator constructs an interference graph, assigns aligned offsets by first fit, and emits JSON, Graphviz, typed C unions/structures, and static assertions. For the v2 `find_uv` annotations, it reproduces the 172,080-byte layout, and the generated header matches the size and offsets of the Arm workspace types.

A coverage check enumerates all 4 declared physical members and 18 direct member or whole-phase access forms; an unannotated member or direct access is rejected. The generator does not infer phase meaning or lifetimes from C, nor does it establish the absence of indirect aliases, pointer escape, missing failure-edge cleanup, function-pointer targets, or optimization-induced changes. Its guarantees therefore depend on annotation completeness and are supplemented by lifetime audit tables, canaries, zeroization tests, and ELF audits.

11 Limitations

The evidence supports a narrower conclusion than an unconstrained “embedded SQIsign” claim:

1. **Target scope.** All physical measurements use one RP2350 board and one Cortex-M33 core. v2 covers Level I/RADIX32. The two-function variant’s host response checks and Arm frames cover all three v3 parameter sets, but target whole-image evidence covers only `p324_3`.
2. **Input scope.** The v2 target result repeats one deterministic KeyGen/Sign/Verify path twice. Its operation-PSP analysis closes linked input-dependent Thread-mode paths for the fixed image, but not handler or MSP behavior. The two-function variant has one target vector and no target timing distribution.

3. **Functional scope.** v2 differential tests and v3 official response checks use finite test sets. The C correspondence audit and proposition 2 do not form an all-input semantics proof.
4. **Manual contracts.** The coverage check catches declared-member and direct-access omissions, but phase meaning, indirect aliases, escape, and failure cleanup remain manually reviewed assumptions.
5. **Stack scope.** A whole-program, interrupt-inclusive maximum requires resolution of handler callbacks, enabled IRQs and priorities, nesting, and live MSP state. The two-function variant additionally requires bounds for its remaining dynamic frames.
6. **Security scope.** The implementations are variable-time. Full-Sign power/EM traces, independent acquisitions, positive-control attack reproduction, masking, and a complete constant-time rewrite were not evaluated and are required before claiming resistance.
7. **Practicality.** v2 KeyGen and Sign are too slow for many products. The 211,072 unreserved bytes may accommodate system integration or countermeasures, but that capacity has not been demonstrated with production RNG, communications, interrupts, and countermeasure state active together.

12 Conclusion

Typed lifetime scheduling and an order-preserving norm sketch reduce the SQIsign v2 Level-I operation arena from 353,008 to 172,080 bytes. The resulting RP2350 firmware uses no GMP, heap, or external PSRAM, reserves 321,408 bytes of SRAM, and leaves 211,072 bytes unreserved. Two fresh-process repetitions of the official-request differential experiment, lifetime and source audit records, linked-image audits, and fixed-image operation-PSP analysis support this bounded result.

In SQIsign v3, three local overlays across two functions reduce all six tested Arm frames over the three official parameter sets. One commit-pinned `p324_3` target path reduces the Sign PSP watermark from 101,060 to 96,396 bytes. This transfer supports the reuse of the lifetime-scheduling method, while the v2 norm-sketch transformation remains version-specific.

The two-function v3 result is limited to local frame analysis and one target path, and the annotation-driven placement depends on manually established lifetimes. Both generations remain variable-time, and neither has analog resistance evidence or an interrupt-inclusive whole-program stack bound. Accordingly, the result concerns auditable memory engineering rather than general optimality or hardened deployment readiness.

Artifact Availability

Source-reconstruction patches, scripts, machine-readable audit tables, firmware digests, and RP2350 execution records are available at <https://github.com/quantumshiro/tinysqisign>.

References

- [1] G. Alagic, M. Bros, P. Ciadoux, Q. Dang, T. H. Dang, J. Kelsey, J. Lichtinger, Y.-K. Liu, C. Miller, D. Moody, R. Peralta, R. Perlner, A. Robinson, H. Silberg, D. Smith-Tone, and N. Waller. *Status Report on the Second Round of the Additional Digital Signature Schemes for the NIST Post-Quantum Cryptography Standardization Process*. Tech. rep. NIST IR 8610. National Institute of Standards and Technology, May 2026. DOI: 10.6028/NIST.IR.8610. URL: <https://doi.org/10.6028/NIST.IR.8610>.
- [2] National Institute of Standards and Technology. *Round 3 Additional Digital Signature Schemes*. 2026. URL: <https://csrc.nist.gov/projects/pqc-dig-sig/round-3-additional-signatures> (visited on 09/04/2026).
- [3] L. De Feo, D. Kohel, A. Leroux, C. Petit, and B. Wesolowski. “SQIsign: Compact Post-Quantum Signatures from Quaternions and Isogenies”. In: *Advances in Cryptology – ASIACRYPT 2020*. Vol. 12491. Lecture Notes in Computer Science. Springer, 2020, pp. 64–93. DOI: 10.1007/978-3-030-64837-4_3. URL: https://doi.org/10.1007/978-3-030-64837-4_3.
- [4] SQIsign submission team. *SQIsign: Algorithm Specifications and Supporting Documentation, Version 2.0.1*. July 2025. URL: <https://sqisign.org/spec/sqisign-20250707.pdf> (visited on 09/04/2026).
- [5] SQIsign submission team. *SQIsign: Algorithm Specifications and Supporting Documentation, Version 3.0*. Tech. rep. National Institute of Standards and Technology, Sept. 2026. URL: <https://sqisign.org/spec/sqisign-20260901.pdf> (visited on 09/04/2026).

- [6] G. J. Chaitin, M. A. Auslander, A. K. Chandra, J. Cocke, M. E. Hopkins, and P. W. Markstein. “Register Allocation via Coloring”. In: *Computer Languages* 6.1 (1981), pp. 47–57. DOI: 10.1016/0096-0551(81)90048-5.
- [7] J. Gergov. “Algorithms for Compile-Time Memory Optimization”. In: *Proceedings of the Tenth Annual ACM-SIAM Symposium on Discrete Algorithms*. ACM/SIAM, 1999, pp. 907–908. DOI: 10.5555/314500.315082.
- [8] W. Kim, J. Lee, H. Kim, and C. Lee. “SQIsign with Fixed-Precision Integer Arithmetic”. In: *Public-Key Cryptography – PKC 2026*. Vol. 16553. Lecture Notes in Computer Science. Earlier version: Cryptology ePrint Archive, Paper 2025/1649. Springer, 2026, pp. 3–30. DOI: 10.1007/978-3-032-26737-5_1. URL: https://doi.org/10.1007/978-3-032-26737-5_1.
- [9] W. Kim, C. Lee, and H. Yoo. “Compact Quaternion Algorithms for SQIsign”. In: *Cryptology ePrint Archive, Paper 2026/1031* (2026). URL: <https://eprint.iacr.org/2026/1031>.
- [10] G. Borin, M. Corte-Real Santos, J. Komada Eriksen, R. Invernizzi, M. Mula, S. Schaeffler, and F. Vercauteren. “Qlapoti: Simple and Efficient Translation of Quaternion Ideals to Isogenies”. In: *Cryptology ePrint Archive, Paper 2025/1604* (2025). URL: <https://eprint.iacr.org/2025/1604>.
- [11] Y.-F. Lai. “Qlapoti+ and More: Optimizing Isogeny-based Signatures”. In: *Cryptology ePrint Archive, Paper 2026/1640* (2026). URL: <https://eprint.iacr.org/2026/1640>.
- [12] M. Duparc, A. Leroux, and S. Schaeffler. “Qlapoty: Improved Analysis and Efficiency for Quaternionic Ideal to Isogeny Transformation”. In: *Cryptology ePrint Archive, Paper 2026/1700* (2026). URL: <https://eprint.iacr.org/2026/1700>.
- [13] M. J. Kannwischer, M. Krausz, R. Petri, and S.-Y. Yang. “pqm4: Benchmarking NIST Additional Post-Quantum Signature Schemes on Microcontrollers”. In: *Cryptology ePrint Archive, Paper 2024/112* (2024). URL: <https://eprint.iacr.org/2024/112>.
- [14] F. Carvalho Rodrigues, D. Gazzoni Filho, G. Adj, I. A. Canales-Martínez, J. Chávez-Saab, J. López, M. Scott, and F. Rodríguez-Henríquez. “Generation of Fast Finite Field Arithmetic for Cortex-M4 with ECDH and SQIsign Applications”. In: *Cryptology ePrint Archive, Paper 2025/1322* (2025). URL: <https://eprint.iacr.org/2025/1322>.
- [15] M. A. Aardal, G. Adj, A. Alblooshi, D. F. Aranha, I. A. Canales-Martínez, J. Chávez-Saab, D. L. Gazzoni Filho, K. Reijnders, and F. Rodríguez-Henríquez. “Optimized One-Dimensional SQIsign Verification on Intel and Cortex-M4”. In: *Cryptology ePrint Archive, Paper 2024/1563* (2024). URL: <https://eprint.iacr.org/2024/1563>.
- [16] L. De Feo, L.-J. Jian, T.-Y. Wang, and B.-Y. Yang. “SQIsign on ARM”. In: *Cryptology ePrint Archive, Paper 2026/394* (2026). URL: <https://eprint.iacr.org/2026/394>.
- [17] Y. Hu, S. Shen, H. Yang, and W. Wang. “Paving the Way for SQIsign: Toward Efficient Deployment on 32-bit Embedded Devices”. In: *Mathematics* 12.19 (2024), p. 3147. DOI: 10.3390/math12193147. URL: <https://doi.org/10.3390/math12193147>.
- [18] W. Wang, C. Wang, Y. Wu, Q. Xue, J. Zheng, and Y. Zhao. “Exposing SIMD Parallelism in SQIsign: An AVX-512 Implementation”. In: *arXiv preprint arXiv:2608.13948* (2026). DOI: 10.48550/arXiv.2608.13948. URL: <https://arxiv.org/abs/2608.13948>.
- [19] Anchorage Digital. *sqisign-rs: A Pure Rust Implementation of SQIsign v2.0*. Repository release 0.5.0. 2026. URL: <https://github.com/anchorageoss/sqisign-rs> (visited on 09/04/2026).
- [20] SQIsign submission team. *The SQIsign Implementation, Version 3.0*. Git tag `nist-v3`; frozen revision 6d017708... Sept. 2026. URL: <https://github.com/SQIsign/the-sqisign/tree/nist-v3> (visited on 09/04/2026).
- [21] F. Kouider, A. Mukherjee, D. Jacquemin, and P. Kutas. “Constant-time Integer Arithmetic for SQIsign”. In: *Cryptology ePrint Archive, Paper 2025/832* (2025). URL: <https://eprint.iacr.org/2025/832>.
- [22] O. Hanyecz, A. Karenin, E. Kirshanova, P. Kutas, and S. Schaeffler. “Constant Time Lattice Reduction in Dimension 4 with Application to SQIsign”. In: *Cryptology ePrint Archive, Paper 2025/027* (2025). URL: <https://eprint.iacr.org/2025/027>.
- [23] A. Basso, C. He, D. Jacquemin, F. Kouider, P. Kutas, A. Mukherjee, S. Schaeffler, and S. Sinha Roy. “Constant-time Quaternion Algorithms for SQIsign”. In: *Cryptology ePrint Archive, Paper 2025/2192* (2025). URL: <https://eprint.iacr.org/2025/2192>.

- [24] A. Mukherjee, M. Czuprynyk, D. Jacquemin, P. Kutas, and S. Sinha Roy. “Simple Power Analysis Attack on SQIsign”. In: *Progress in Cryptology – AFRICACRYPT 2025*. Vol. 15651. Lecture Notes in Computer Science. Earlier version: Cryptology ePrint Archive, Paper 2025/830. Springer, 2025, pp. 245–269. DOI: 10.1007/978-3-031-97260-7_12. URL: https://doi.org/10.1007/978-3-031-97260-7_12.
- [25] T. Granlund and the GMP development team. *GNU MP: The GNU Multiple Precision Arithmetic Library*. 6.3.0. 2023. URL: <https://gmplib.org/gmp-man-6.3.0.pdf> (visited on 09/04/2026).
- [26] Raspberry Pi Ltd. *RP2350 Datasheet: A Microcontroller by Raspberry Pi*. RP-008373-DS-2. 2025. URL: <https://pip-assets.raspberrypi.com/categories/1214-rp2350/documents/RP-008373-DS-2-rp2350-datasheet.pdf?disposition=inline> (visited on 09/04/2026).
- [27] J. Yiu. *How Much Stack Memory Do I Need for My Arm Cortex-M Applications?* Arm. Mar. 2016. URL: <https://developer.arm.com/community/arm-community-blogs/b/architectures-and-processors-blog/posts/how-much-stack-memory-do-i-need-for-my-arm-cortex-m-applications> (visited on 09/04/2026).
- [28] T. Schneider and A. Moradi. “Leakage Assessment Methodology: A Clear Roadmap for Side-Channel Evaluations”. In: *Journal of Cryptographic Engineering* 6.2 (2016), pp. 85–99. DOI: 10.1007/s13389-016-0120-y. URL: <https://eprint.iacr.org/2015/207>.